

Operational Governance Framework: Autonomous Memory Lifecycles and Consolidation Protocols

1. Foundations of Autonomous Governance

In the engineering of cognitive infrastructure, the "crucial correction" to legacy agent design is the formal unbundling of representation, continuous processing, and backend generation. By decoupling the cognitive state from the underlying LLM, we resolve the "prompt-exit dormancy" that plagues reactive chatbots. This framework establishes a governance layer where the backend acts as a downstream consumer of a living internal state. This ensures that the agent maintains situated awareness through an event-coded model, preserving a clear evolutionary path toward a natively tokenizerless, resonant architecture. | Feature | Legacy Conflated Architecture | Unbundled Resonant Architecture || ----- | ----- | ----- || **Logic Flow** | Upstream-driven (Prompt $\rightarrow$ Response) | Downstream-driven (Sensors $\rightarrow$ State $\rightarrow$ Backend) || **State** | Ephemeral / Stateless / Volatile | Living internal state via Ganglion/CA Bus || **Representation** | Standard Subword Tokenization | Event Codec: Resonant segmentation, hashed feature vectors, and lossless byte IDs || **Backend Role** | Primary logic and state engine | Optional renderer (Echo local or HF remote) || **Memory** | Reactive context windows | Append-only, non-destructive substrate | This architectural realignment shifts the system from a stateless utility to a modular AI creature with a persistent lifecycle and formalized knowledge integrity.

2. The Tiered Memory Hierarchy and State Integrity

Strategic governance requires a rigorous distinction between episodic sensations and durable takeaways. Without a tiered hierarchy, the computational overhead of raw perception leads to system collapse. By separating the noise of raw sensory input from distilled internal beliefs, we enable the agent to maintain focus on long-term objectives while remaining responsive to immediate environmental shifts.

The Memory Store: Non-Destructive Substrate

The primary storage layer is an append-only substrate. While raw turns are logged for episodic context, the Long-Term Memory (LTM) is reserved for validated takeaways. To maintain state integrity, all LTM entries must be formatted with specific metadata for search and salience ranking:

- **Kind:** Categorical type (e.g., belief, task, thread, reflection).
- **Strength:** Numerical weight indicating the entry's current influence.
- **Use Count:** Frequency of retrieval, driving reinforcement or decay.
- **Last-Accessed Time:** Timestamp for temporal relevance analysis.
- **Evidence:** Supporting data strings or observations validating the entry.
- **Source IDs:** Lossless byte IDs linking back to original episodic events.

The Ganglion and CA Bus: Temporal Continuity

The "Ganglion" serves as the system's continuously ticking substrate, preventing the agent from entering a dormant state between prompts. Event signatures are written to the "CA Bus" as compact patches. This bus is designed to shift, update, and bleed between channels at every tick, facilitating a non-linear state evolution. This temporal continuity ensures the

agent is constantly "noticing" and processing internal state changes independently of external user triggers. This hierarchy establishes the data layer required for active maintenance cycles to govern the movement of information.

3. The Sleep Cycle: Formalized Promotion and Decay Protocols

The "Sleep" pass is the core maintenance protocol for filtering high-priority active memories from the noise of sensory saturation. This process is essential for cognitive de-cluttering, ensuring that the active memory store remains performant while core identity-like items are reinforced and protected from accidental decay.

The /sleep Procedural Manual (SOP)

The /sleep command invokes a multi-stage consolidation routine:

1. **Execute Promotion Mechanics:** Identify journal summaries (tasks, beliefs, threads) and episodic memories with high strength/salience. Promote verified entries into the durable LTM store.
2. **Apply Identity Protection:** Hard-gate all "protected identity-like items" (e.g., self-model, core names, persistent goals). These items must bypass decay logic to prevent the agent from losing its self-model.
3. **Perform Reinforcement Logic:** Scan existing LTM entries for recurring observations. Rather than creating duplicates, increment the "Strength" and "Use Count" of existing records to consolidate the belief system.
4. **Initiate Decay and Archival:** Evaluate LTM entries with low "Use Count" and stale "Last-Accessed" timestamps. Identify records falling below the minimum strength threshold and move them to a non-active archive.

Critical Takeaways of the Sleep Pass

1. **Durable Distinction:** Establishes the boundary between "what happened recently" and "what should still matter later."
2. **Capacity Management:** Prevents "store bloat" by archiving low-priority records once the active substrate reaches its threshold.
3. **Self-Model Preservation:** Protects core beliefs and identity-anchors from the decay cycles that purge episodic noise. Successful consolidation through the Sleep cycle provides the stable foundation necessary for associative recombination in the Dream cycle.

4. The Dream Cycle: Associative Rehearsal and Synthetic Linkage

The "Dream" cycle is a strategic pass designed to prevent knowledge from becoming stagnant. By fostering synthetic adjacency, the system recombines disparate long-term memories to find new connections, enabling internal state evolution without the risk of unconstrained hallucination.

The /dream Protocol

The /dream pass executes associative rehearsal. It scans LTM entries to identify overlaps in evidence or source IDs, creating "guarded synthetic links" between adjacent memories.

These links function as **imaginal sidesteps**, allowing the agent to suggest new relationships between tasks or beliefs. This process provides the internal pressure signals that will later drive the Resonance Field during retrieval. **Design Stance: Rehearsal vs.**

Hallucination The Dream cycle is a structured recombination of existing data. It does not engage in unconstrained model "hallucination"; rather, it creates guarded links within the existing memory architecture to foster synthetic internal state evolution. This internal rehearsal ensures that subsequent external responses are informed by a highly interconnected, distilled knowledge web.

5. Resonant Gating and Retrieval Policy

To govern how knowledge is surfaced, the system utilizes a "Resonance Field." This acts as an internal pressure signal for ranking knowledge based on salience and active goals. This prevents the "empty naming" common in standard RAG systems, ensuring that retrieved data rises to the surface because it is contextually resonant.

Normal Context Recall Policy

The retrieval policy is backend-agnostic, allowing the system to nudge various renderers (Echo or HF) toward durable beliefs:

- **Short-Term Working Memory (STM):** Provides the immediate conversational context.
- **Long-Term Memory (LTM) Hits:** Surfaces distilled takeaways and entrenched beliefs.
- **Synthetic Signal:** Integrates links generated during the Dream cycle to influence response direction.

Impact on Cognitive Performance

By using resonance as a gating signal, the agent exhibits situated awareness. It avoids the reactive "waiting for prompt" loop by allowing its internal self-model and goals to exert pressure on the output. However, this high-level cognitive governance must be tempered by the specific hardware constraints of the deployment environment.

6. Hardware-Aware Governance and Operational Constraints

Cognitive governance is ineffective if the underlying hardware fails to meet latency targets. In a Dual NVIDIA Tesla P40 (Pascal) environment, the SM_61 architecture dictates specific operational truths. Because Pascal lacks tensor cores and suffers from weak FP16 performance, governance must favor GGUF quantized models via llama.cpp to ensure reliable inference.

Device Routing and Maintenance Checklist

- **Inference Mapping:** Utilize GGUF Q4/Q5/Q8 quants via llama.cpp. Avoid raw FP16 HF models.
- **ASR/TTS Routing:** Pin Audio Speech Recognition (ASR) to cuda:0. Offload TTS and image captioning to the CPU to preserve VRAM for the Qwen language brain.
- **NUMA Node Affinity:** Execute `nvidia-smi topo -m` to verify PCIe lane ownership. Pin CPU-side inference threads to the local NUMA node using `numactl` to mitigate QPI/UPI latency.
- **Persistence Mode:** Enable `nvidia-persistenced` to prevent cards from dropping power states, which causes severe latency and bus-vanishing errors.
- **Thermal Regulation:** Cap P40 cards at **200W** (`nvidia-smi -pl 200`) to prevent thermal throttling in passively cooled chassis environments.

- **Driver Locking:** Pin the environment to the **580 branch** (the final Pascal-supporting branch) to prevent unsupported driver updates from breaking the stack.

Operational Summary

The goal of this framework is to deploy a cognitive system that is functional today while meticulously preserving the path to a natively event-coded, tokenizerless model. By unbundling the architecture, enforcing a tiered memory lifecycle, and respecting the specific topographic constraints of the Pascal hardware, we build an agent capable of long-term evolutionary growth and grounded reasoning.